2. Literature Survey

2.1 Introduction

The ability to learn continuously from an ever-evolving data stream — retaining previously acquired knowledge while integrating new information — is a hallmark of biological intelligence. Human beings effortlessly accumulate skills over a lifetime without losing prior mastery. Artificial neural networks, however, exhibit a striking failure mode: when trained sequentially on multiple tasks, they abruptly lose performance on earlier tasks as they adapt to new ones. This phenomenon is known as catastrophic forgetting (also termed catastrophic interference).

Catastrophic forgetting was first rigorously identified by McCloskey & Cohen (1989) and extensively studied by French (1999), who showed that the problem is rooted in the way distributed representations share the same weight space across tasks. Since then, the field of Continual Learning (CL) — also called Lifelong Learning or Incremental Learning — has produced three broad families of solutions: regularization-based methods, replay-based methods, and architecture-based methods. Recent research shows that hybrid combinations of these strategies consistently outperform any single approach.

This literature survey traces the intellectual history of these approaches, beginning with the foundational memory mechanisms of neural network architectures, through the mathematical characterization of forgetting, and ending with a detailed review of the most influential methods in each category.

2.2 Memory Mechanisms in Neural Networks and Transformers

To understand why neural networks forget, one must first understand how they remember. Unlike traditional computer memory — where information is stored in discrete, addressable registers — neural networks encode knowledge in a distributed manner across millions of numerical parameters. When a network learns to classify images, the resulting knowledge is spread as a pattern across the entire parameter space, not localized to any single weight.

2.2.1 Distributed Representations

Knowledge stored in neural networks has three key characteristics that together give rise to catastrophic forgetting when sequential training occurs:

- Superimposition: Multiple pieces of information share the same weights simultaneously, enabling rich generalization.
- Interference-prone: Modifying weights for new learning can corrupt existing information encoded in the same weights.

- Non-localized: A single concept activates patterns across many neurons, so there is no simple memory address one can protect.

This distributed representation is both a strength — enabling generalization and transfer — and a fundamental weakness when tasks must be learned sequentially.

2.2.2 The Transformer Architecture and Self-Attention

The Transformer architecture, introduced by Vaswani et al. (2017), revolutionized deep learning by relying entirely on self-attention mechanisms. Understanding the Transformer's memory is important because Vision Transformers (ViT) are now widely used in continual learning research, and their mechanisms of forgetting differ subtly from convolutional networks.

Self-attention computes relationships between all positions in a sequence through three learned projection matrices — Query (Q), Key (K), and Value (V). For each token, the mechanism asks: "how relevant is every other token to me?" using dot-product similarities, scales them to prevent gradient saturation, and takes a weighted average of the value vectors. The result is an output that dynamically focuses on the most relevant context.

Formula: Scaled Dot-Product Attention

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Variable Breakdown:

- ***Q (Query)*** — "What am I looking for?" — the current token's learned projection requesting relevant information.
- ***K (Key)*** — "What do I offer?" — representations describing what each token contains.
- ***V (Value)*** — The actual content to be retrieved and aggregated.
- $\boldsymbol{QK^T}$ — Dot product measuring similarity between all query-key pairs.
- $\sqrt{\boldsymbol{d_k}}$ — Scaling factor (square root of key dimension) preventing extreme dot-product values that would saturate the softmax.
- ***softmax*** — Converts similarity scores into a probability distribution summing to 1.

Key Findings:

- Multi-head attention applies this mechanism H times in parallel, allowing joint attention to different representation subspaces.
- Feed-forward layers within the Transformer act as key-value memories storing factual associations (Geva et al., 2021).
- Scialom et al. (2022) confirmed that even large instruction-tuned Transformers suffer significant forgetting during sequential fine-tuning, motivating explicit CL strategies for ViT-based backbones.

2.3 Catastrophic Forgetting: The Fundamental Challenge

2.3.1 The Stability-Plasticity Dilemma

The core tension in continual learning is captured by Grossberg's (1980) stability-plasticity dilemma. Plasticity refers to the network's ability to rapidly absorb new information; stability refers to its ability to retain previously learned knowledge without degradation. High plasticity enables fast learning but risks overwriting old memories; high stability preserves old knowledge but may prevent adaptation. No simple training procedure simultaneously achieves both — this is the fundamental challenge that the continual learning literature seeks to resolve.

2.3.2 The Gradient Update Mechanism of Forgetting

When a network with parameters θ is trained sequentially on tasks T_1 through T_n, each gradient update step is computed purely with respect to the current task's loss, without regard for previously learned optima. The standard update rule moves the parameter vector toward a minimum of the new task's loss:

Formula: Gradient Descent Update (causes forgetting)

$$\theta^* = \theta - \eta \nabla_\theta \mathcal{L}_B(\theta)$$

Variable Breakdown:

- ***θ*** — Current model parameters, previously optimized for Task A.
- ***η (eta)*** — Learning rate — controls the step size in the gradient direction.
- ***∇θ ℒB(θ)*** — Gradient of Task B's loss with respect to all parameters — computed purely for Task B, ignoring Task A.
- ***θ**** — Updated parameters after the gradient step — now optimized for Task B, potentially far from the Task A optimum.

The gradient ∇θ ℒB(θ) is computed purely for Task B. If the loss landscapes of Tasks A and B are misaligned — which is the rule rather than the exception — the new optimum will be far from the old one, and performance on Task A collapses. This is the mathematical root of catastrophic forgetting.

2.3.3 Quantifying Forgetting: Evaluation Metrics

To measure forgetting empirically and compare methods fairly, the CL field has standardized on four complementary metrics. Together they provide a multi-dimensional view of a model's behavior across the stability-plasticity spectrum.

Metric 1: Individual Task Forgetting

$$F_i = max_{j<t} a_{j,i} - a_{t,i}$$

- F_i — Forgetting on task i — how much worse we are compared to our peak performance on that task.
- $max_{j+t}\ a_{j,i}$ — Best accuracy ever achieved on task i during the learning sequence.
- $a_{t,i}$ — Current accuracy on task i after learning up through task t.

Metric 2: Average Accuracy (AA)

$$AA = \frac{1}{T}\sum_{i=1}^{T} a_{T,i}$$

- T — Total number of tasks learned.
- $a_{t,i}$ — Accuracy on task i after learning all T tasks. Higher AA = better overall retention.

Metric 3: Aggregate Forgetting Measure (F)

$$F = \frac{1}{T-1}\sum_{i=1}^{T-1}\left(max_{j<T}\, a_{j,i} - a_{T,i}\right)$$

This averages the peak-to-final performance drop across all non-final tasks. A value of 0 means perfect retention; positive values indicate forgetting. This is the primary metric used to compare methods in this project.

Metric 4: Backward Transfer (BWT)

$$BWT = \frac{1}{T-1}\sum_{i=1}^{T-1}\left(a_{T,i} - a_{i,i}\right)$$

- $a_{t,i}$ — Final accuracy on task i after all training.
- $a_{i,i}$ — Accuracy on task i right after initially learning it.

Negative BWT indicates forgetting (performance declined after initial learning). Positive BWT is rare and desirable, indicating that later learning improved earlier task performance.

Metric 5: Forward Transfer (FWT)

$$FWT = \frac{1}{T-1}\sum_{i=2}^{T}\left(a_{i-1,i} - b_i\right)$$

- $a_{i-1,i}$ — Zero-shot performance on task i before training on it — benefit from prior tasks.
- b_i — Baseline performance from a randomly initialized model (no prior knowledge).

Positive FWT means prior knowledge gave a head start on new tasks. It measures the benefit of transfer learning across the task sequence.

Key Insight:

- AA measures absolute performance; F and BWT measure forgetting; FWT measures transfer.
- A good CL method should maximize AA and FWT while minimizing |F| and driving BWT toward zero or positive.
- All methods in this project are evaluated on all five metrics, providing a complete picture.

2.3.4 Continual Learning Scenarios

Van de Ven & Tolias (2019) formalized three distinct evaluation scenarios that differ in information available at test time, representing increasing levels of difficulty:

- Task-Incremental Learning (Task-IL): Task identity is provided at test time. The model selects the correct output head for the given task. Easiest scenario.
- Domain-Incremental Learning (Domain-IL): Task identity not provided; the input distribution shifts across tasks but the output space is fixed. Medium difficulty. Instantiated in this project by Permuted MNIST.
- Class-Incremental Learning (Class-IL): Task identity not provided; new classes are added each task; the model must classify across all seen classes simultaneously. Most challenging and most realistic. Primary scenario for CIFAR-100, Mini-ImageNet, and Tiny-ImageNet in this project.

2.4 Regularization-Based Methods

Regularization-based methods address catastrophic forgetting by modifying the loss function to penalize changes to parameters that were important for previous tasks. They require no additional storage of old data — all protection is achieved through mathematical constraints on the optimization trajectory. This family was the first principled approach to continual learning and remains influential both as standalone baselines and as components in hybrid systems.

2.4.1 Elastic Weight Consolidation (EWC)

Kirkpatrick et al. (2017) introduced Elastic Weight Consolidation, one of the most influential works in continual learning. The key insight is that not all weights are equally important: some can be freely modified without affecting task performance, while others are critical. EWC estimates parameter importance using the Fisher Information Matrix (FIM), which measures the curvature of the loss surface — a high-curvature direction indicates that small perturbations to that parameter cause large increases in loss, signaling importance.

Rather than protecting all weights equally (as L2 regularization does), EWC places stronger constraints on high-importance weights and allows free movement of low-importance ones, creating a mathematically grounded elastic anchor.

Formula: EWC Modified Loss

$$\mathcal{L}(\theta) = \mathcal{L}_B(\theta) + \sum_i \frac{\lambda}{2} F_i \left(\theta_i - \theta^*_{A,i}\right)^2$$

Variable Breakdown:

- $\mathcal{L}B(\theta)$ — Standard cross-entropy loss for the new task B — ensures new learning proceeds.
- F_i — Diagonal Fisher Information for parameter i — measures how "important" this weight was for Task A. Computed as the variance of the gradient: $F_i = \hat{Z}[\partial \log p(y|x,\theta)/\partial\theta_i]^2$.
- λ — Regularization strength. Higher λ = stronger protection for old tasks, but potentially slower adaptation to new tasks.
- $\theta^*_{a,i}$ — Optimal parameter i after training on Task A — the anchor we protect against deviation.
- $(\theta_i - \theta^*_{a,i})^2$ — Squared deviation from old optimum — penalizes parameters for straying far from Task A configuration.

Key Findings:

- Successfully maintained performance on multiple Atari games and classification tasks simultaneously — a landmark result confirming selective weight protection as viable.
- The diagonal Fisher approximation is efficient but may miss important cross-weight correlations.
- Importance must be recomputed at each task boundary, adding overhead that grows with the number of tasks.

2.4.2 Synaptic Intelligence (SI)

Zenke, Poole & Ganguli (2017) proposed Synaptic Intelligence, which accumulates parameter importance online during the training trajectory rather than computing it only at task boundaries. Unlike EWC, SI captures the full learning dynamics — not just the endpoint — making it more sensitive to weights that were active throughout training.

Formula: SI Online Importance Accumulation

$$\Omega_k = \sum_{\mu < t} \frac{\omega_k^{\mu}}{\left(\Delta_k^{\mu^2} + \xi\right)}$$

Variable Breakdown:

- $\boldsymbol{\Omega_k}$ — Total accumulated importance of weight k across all previous tasks. Used to protect important weights via a penalty term analogous to EWC.
- $\boldsymbol{\omega_k^{\mu}}$ — Path integral contribution: how much did weight k help reduce the loss during task μ? Computed as the gradient-weighted path integral over all update steps.
- $\boldsymbol{\Delta_k^{\mu}}$ — Total change in weight k during task μ. Larger changes indicate more flexible weights that are less critical.
- $\boldsymbol{\xi}$ — Small constant (e.g., 0.1) preventing division by zero.

Key Insight & Findings:

- A weight is important if it contributed much to learning (high ω) while barely changing (low Δ). This captures weights that are "load-bearing" in the loss landscape.
- Unlike EWC, SI updates importance continuously during training, capturing the full learning trajectory rather than just the task-end snapshot.
- Achieves comparable or better results than EWC on Split-MNIST while being more computationally efficient — no separate FIM computation step required.

2.4.3 Learning without Forgetting (LwF)

Li & Hoiem (2017) introduced Learning without Forgetting, which uses knowledge distillation as the mechanism for protecting old task knowledge. Before learning a new task, LwF records the model's predictions on the new task data as "pseudo-labels" for old tasks. During training, the loss combines a standard cross-entropy term with a distillation term that penalizes deviation from these pseudo-labels.

Formula: LwF Combined Loss

$$\mathcal{L} = \mathcal{L}_{new}\left(Y_{\mathrm{n}}, \hat{Y}_{\mathrm{n}}\right) + \lambda \cdot \mathcal{L}_{distill}\left(Y_{\mathrm{o}}, \hat{Y}_{\mathrm{o}}\right)$$

Variable Breakdown:

- $\boldsymbol{\mathscr{L}_{new}(Y_n, \hat{Y}_n)}$ — Standard cross-entropy loss for the new task — ground truth labels Y_n vs. predictions $\hat{Y}_n$.
- $\boldsymbol{\mathscr{L}_{distill}(Y_o, \hat{Y}_o)}$ — Distillation loss for old tasks — measures deviation of current outputs from pre-task outputs recorded as pseudo-labels.

- Y_o — Old task outputs recorded from the model before training begins — serve as pseudo-labels.
- λ — Balance hyperparameter between new task learning and old knowledge preservation.

Key Findings:

- No old data required — the model's own predictions on new task inputs serve as proxies for old-task knowledge.
- Performs comparably to joint training when tasks are similar; degrades as tasks become more dissimilar (pseudo-labels become unreliable).
- The knowledge distillation component of LwF has been incorporated into many subsequent hybrid methods, including DER and iCaRL.

2.5 Replay-Based Methods

Replay-based methods combat forgetting by maintaining a memory buffer of examples from previous tasks and interleaving them with new task data during training. By re-exposing the model to past data, the optimizer is forced to maintain gradients compatible with old tasks. Replay methods are more flexible than regularization approaches because they can encode arbitrarily complex old-task structure.

2.5.1 Gradient Episodic Memory (GEM)

Lopez-Paz & Ranzato (2017) introduced Gradient Episodic Memory, which stores a small episodic memory for each previous task and uses it to constrain gradient updates. At each training step, GEM ensures that the chosen gradient update does not increase the loss on any previously seen task — a principled safety guarantee.

Formula: GEM Constrained Optimization

$$min\ \frac{1}{2} \| g - \bar{g} \|^2\ s.t.\ \langle \bar{g}, g_k \rangle \geq 0\ \forall k < t$$

Variable Breakdown:

- g — Raw gradient for the current new task — what we ideally want to apply.
- $\bar{g}$ ***(g-tilde)*** — Modified gradient we will actually apply — must satisfy old-task constraints.
- $\| g - \bar{g} \|^2$ — L2 distance between desired and actual gradient — the objective is to minimize this (stay as close to desired as possible).
- $\langle \bar{g}, g_k \rangle \geq 0$ — Constraint for each old task k: the inner product between the modified gradient and old-task gradient must be non-negative, ensuring we do not oppose old-task learning.

Key Findings:

- GEM alleviates forgetting while enabling beneficial backward transfer — a unique property among CL methods.
- Outperforms fine-tuning and regularization baselines on MNIST and CIFAR-100 variants.
- Limitation: solving a quadratic program at each update step is computationally expensive, motivating A-GEM.

2.5.2 Averaged GEM (A-GEM)

Chaudhry et al. (2019) proposed Averaged GEM, which simplifies GEM by replacing per-task gradient constraints with a single constraint based on the averaged gradient from a random sample of the replay buffer. This reduces the quadratic program to a simple projection, making A-GEM nearly as efficient as standard gradient descent while matching GEM-level performance.

Key Findings:

- Achieves one of the best accuracy-efficiency trade-offs among replay methods.
- The projection operation replaces a QP solver, reducing per-step overhead from $O(T^2)$ to $O(1)$.
- Used as a primary baseline in this project due to its practical deployability.

2.5.3 Dark Experience Replay (DER)

Buzzega et al. (2020) introduced Dark Experience Replay, which enriches the replay mechanism by storing not just input-label pairs but also the model's logit vectors (soft outputs) at the time of storage. When samples are replayed, the model is penalized not for deviating from hard labels, but for deviating from the complete logit distribution — preserving the full geometry of the learned embedding space.

Formula: DER Loss Function

$$\mathcal{L}_{DER} = \mathcal{L}_{CE}(f_\theta(x), y) + \alpha \| f_\theta(x_{mem}) - z_{mem} \|^2$$

Variable Breakdown:

- $\mathcal{L}^{cl}(f\theta(x), y)$ — Cross-entropy loss on current task data — standard classification training.
- x^{mem} — Sample retrieved from the replay buffer (stored from a previous task).
- z^{mem} — The model's output logit vector when x^{mem} was stored — the "dark experience."
- $\| f\theta(x^{mem}) - z^{mem} \|^2$ — MSE penalizing the model's current predictions for deviating from stored predictions.
- α — Weight balancing new task learning against old knowledge preservation via logit matching.

Why "Dark" Experience?

- A hard label says "cat." A stored logit vector says "88% cat, 10% dog, 2% tiger" — encoding class relationships that hard labels discard.
- This richer signal provides stronger, more informative gradients for preserving old knowledge.
- DER consistently outperforms methods that rely only on hard labels, especially in realistic scenarios where task boundaries are unclear.

2.5.4 Extended DER (X-DER)

Boschini et al. (2022) extended DER to the Class-Incremental setting with X-DER, adding two critical capabilities: (i) revision of stored logits when the classifier head expands to accommodate new classes — ensuring old samples retain meaningful class probabilities as the output space grows; and (ii) a mechanism to prepare the model for learning future unseen classes by reserving output capacity.

Key Findings:

- Achieves state-of-the-art results on CIFAR-100 and Mini-ImageNet for Class-IL.
- Logit revision prevents the stored dark experience from becoming stale as the class space grows — a critical fix for the Class-IL scenario.
- X-DER is the strongest replay-distillation hybrid and the primary benchmark for novel hybrid combinations in this project.

2.5.5 ER-Reservoir

Chaudhry et al. (2019, "On Tiny Episodic Memories") studied the simplest form of experience replay: random reservoir sampling, which maintains a fixed-size buffer by replacing old samples uniformly at random. Despite its simplicity, ER-Reservoir proves to be a surprisingly competitive baseline, often matching or exceeding more complex methods when the buffer size is sufficiently large.

Key Findings:

- Competitive performance at large buffer sizes challenges the value of algorithmic complexity in replay methods.
- Serves as the "simple baseline" reference in this project — the critical question being: how much does additional algorithmic complexity actually add over random replay?

2.6 Architecture-Based Methods

Architecture-based methods avoid forgetting by modifying the network's structure so that different tasks are handled by different (possibly overlapping) subsets of parameters. By

isolating task-specific computation, these methods can theoretically guarantee zero forgetting — at the cost of increased model size or reduced future expressiveness.

2.6.1 Progressive Neural Networks (PNN)

Rusu et al. (2016) introduced Progressive Neural Networks, which add a new "column" (a full copy of the base network) for each new task, while freezing all previous columns. Lateral connections between columns allow the new task to read from previously learned representations, enabling positive forward transfer. Previous columns are completely frozen, guaranteeing zero catastrophic forgetting by design.

Formula: Progressive Network Activation

$$h_i^{(k)} = f\left(W_i^{(k)} h_{i-1}^{(k)} + \sum_{j<k} U_i^{(k:j)} h_{i-1}^{(j)}\right)$$

Variable Breakdown:

- $h_i^{(k)}$ — Hidden activation at layer i for task k's column — the output of the k-th sub-network at layer i.
- $W_i^{(k)}$ — Weight matrix for layer i in task k's column — standard feedforward weights, trained freely.
- $U_i^{(kIT)}$ — Lateral connection weights from column j to column k at layer i — imports knowledge from previous tasks.
- Σ_{j+}^{k} — Sum over all previous task columns — allows full access to all previously learned representations.

Key Findings:

- Immune to catastrophic forgetting by construction — frozen columns cannot be modified.
- Enables positive forward transfer — new tasks can leverage all prior learned representations.
- Critical limitation: model size grows linearly with the number of tasks, making PNN impractical for long task sequences.

2.6.2 PackNet

Mallya & Lazebnik (2018) proposed PackNet, which repurposes existing network capacity rather than adding new parameters. After learning each task, PackNet prunes the weights least important to that task and freezes the remaining weights (encoding the task's knowledge). The freed (pruned) capacity is then available for future tasks.

Key Findings:

- Eliminates forgetting by construction (frozen weights cannot change) and is highly parameter-efficient vs. Progressive Networks.
- Available capacity for new tasks decreases over time — performance on later tasks degrades as the fixed network fills up.
- Works well for small numbers of tasks; degrades as the task sequence grows longer.

2.7 Hybrid Methods: Combining Strategies for Superior Performance

The consensus from the CL literature is that no single-strategy approach is universally optimal. Regularization methods are efficient but cannot fully protect complex knowledge. Replay methods are powerful but require memory storage. Architecture methods guarantee zero forgetting but require growing models. Hybrid methods combine multiple strategies to achieve better trade-offs across all dimensions — and represent the central focus of this project.

2.7.1 iCaRL: Incremental Classifier and Representation Learning

Rebuffi et al. (2017) introduced iCaRL, one of the most carefully engineered hybrid methods for Class-Incremental Learning. It combines three components working in concert:

- Herding-based exemplar selection: Stores samples closest to each class's mean embedding (rather than random samples) — maximizing information per stored sample.
- Knowledge distillation: Follows the LwF approach to preserve old task outputs on new data — no separate old-task replay needed for the distillation term.
- Nearest-mean-of-exemplars classifier: Classifies by comparing query embeddings to mean exemplar embeddings per class — a non-parametric decision boundary that adapts gracefully as new classes arrive.

Key Findings:

- Was one of the first methods to demonstrate competitive Class-IL performance.
- Herding selection outperforms random replay at small buffer sizes by extracting more distributional information per stored sample.
- The combination of replay, distillation, and prototype-based classification has influenced the design of numerous subsequent methods.

2.7.2 Progress & Compress

Schwarz et al. (2018) combined architecture-based protection with EWC regularization. The framework maintains two networks: an "active" column trained freely on each new task (Progress phase), and a knowledge base that stores compressed knowledge from all previous tasks via EWC (Compress phase). After learning, the active column is distilled into the knowledge base using EWC consolidation.

Key Findings:

- The separation of learning and consolidation phases avoids the stability-plasticity conflict during training.
- Outperforms EWC alone — the active column can learn freely without regularization constraints, then knowledge is merged post-hoc.
- Influenced later hybrid designs that explicitly decouple the learning and consolidation stages.

2.7.3 De Lange et al. Continual Learning Survey (2021)

De Lange et al. (2021) published the most comprehensive survey of CL methods, evaluating 11 approaches across multiple benchmarks. Their central finding was that hybrid methods combining regularization with replay consistently outperform single-strategy approaches, and the performance gap widens as task sequences grow longer.

Critical Gaps Identified:

- No standardized evaluation protocols across papers — results are rarely directly comparable.
- Inconsistent dataset and backbone choices — confounds between methods and experimental design.
- Near-complete absence of systematic comparisons of hybrid combinations under controlled conditions.

These three gaps directly motivate the comparative study presented in this report.

2.7.4 Knowledge Distillation as a Universal Tool

Hinton, Vinyals & Dean (2015) introduced knowledge distillation for model compression, but it has since become a cornerstone technique for continual learning. The teacher is the model before learning a new task; the student is the model after. The temperature parameter T softens the probability distributions, revealing "dark knowledge" — subtle preferences among non-winning classes that encode rich relational structure.

Formula: Knowledge Distillation Loss

$$\mathcal{L}_{KD} = T^2 \cdot KL\left(\sigma\left(\frac{z_s}{T}\right) \parallel \sigma\left(\frac{z_t}{T}\right)\right)$$

Variable Breakdown:

- z_s — Student network's raw logit outputs (pre-softmax).
- z_t — Teacher network's raw logit outputs — the "old" model we want to preserve.

- ***T*** — Temperature parameter. Higher T produces softer distributions, revealing more class relationship information. Typical values: 2–20.
- ***σ(z/T)*** — Softmax with temperature T — flattens the distribution to expose non-dominant class similarities.
- ***KL(P ‖ Q)*** — Kullback-Leibler divergence measuring how much distribution P differs from Q.
- ***T^2*** — Scaling factor compensating for the gradient magnitude reduction caused by temperature scaling.

Key Findings:

- At T=1 (normal softmax), the dominant class drowns out others. Higher T reveals that "cats look more like dogs than like tables" — encoding class geometry that hard labels discard.
- Knowledge distillation is a component in LwF, DER, X-DER, iCaRL, and Progress & Compress, making it the most widely applicable tool in the hybrid CL toolkit.
- In the continual learning context, distillation preserves the full geometry of the learned embedding space, not just individual class labels.

2.7.5 Continual Learning in Large Language Models

Scialom et al. (2022) studied continual learning in instruction-tuned large language models (Continual-T0), showing that even at scale, catastrophic forgetting remains a significant concern. Models fine-tuned sequentially on 70 diverse datasets suffer substantial performance degradation on earlier tasks. This finding directly motivates the inclusion of ViT-Small as a backbone in this project: Transformer-based architectures, despite their larger capacity, are not immune to forgetting and require explicit continual learning strategies.

Key Findings:

- Self-supervised pretraining provides only partial protection against forgetting — it is not a sufficient solution.
- Continual learning capability emerges from self-supervision pretraining in large models, but degrades rapidly during sequential fine-tuning without explicit CL mechanisms.

2.8 Gap Analysis and Motivation for This Project

Despite the substantial progress surveyed above, three critical gaps remain in the continual learning literature that this project directly addresses:

1. Absence of controlled comparison: While individual hybrid methods have been proposed and evaluated, no study has systematically compared multiple hybrid combinations under identical, controlled conditions using the same backbone architectures, datasets, preprocessing pipelines, and evaluation protocols. Reported results across papers are rarely directly comparable due to these confounds.

2. No practical decision framework: The field lacks a quantitative, data-driven framework for selecting the appropriate continual learning method based on practical resource constraints. A practitioner must choose between methods differing in memory, compute, and performance — but no principled tool exists for this choice.
3. Uncharacterized hybrid interactions: The interaction effects between different hybrid components (e.g., does EWC added to replay help more or less than distillation?) have not been systematically studied through ablation experiments. Which combinations are synergistic and which are redundant remains an open question.

The present project directly addresses all three gaps through a rigorous comparative experimental study, comprehensive ablation analysis, and the development of a data-driven Decision Support Framework.

3. Novelty and Contributions of This Project

While the literature survey above covers methods that have been individually proposed and studied, this project's novelty lies in the systematic, controlled, and resource-aware comparison of these methods, combined with the development of a practical Decision Support Framework. This section enumerates the specific contributions that distinguish this work from existing research.

3.1 Comprehensive and Controlled Comparison

Existing studies typically compare two or three methods on one or two datasets using a single backbone architecture. This project evaluates:

- 5 baseline methods: FineTune, Joint Training, EWC, A-GEM, LwF
- 7 hybrid methods: DER, X-DER, iCaRL, ER+EWC, Progress & Compress, A-GEM+Distill, SI+DER
- 4 benchmark datasets across Domain-IL and Class-IL scenarios at varying scales
- 2 backbone architectures: Slim ResNet-18 (convolutional) and ViT-Small (Vision Transformer)

All experiments share the same preprocessing pipeline, training harness, random seeds, and evaluation protocol — making results directly comparable across methods for the first time.

3.2 Novel Hybrid Combinations

Beyond re-implementing known methods, this project introduces and evaluates three novel hybrid combinations not systematically studied in prior work:

- ER + EWC: Combines reservoir-based experience replay with Elastic Weight Consolidation. The replay buffer addresses distribution shift; EWC provides additional weight-level regularization. The key question — whether these are synergistic or redundant — has not been characterized.
- A-GEM + Distillation: Augments A-GEM's gradient projection with a knowledge distillation term. The constraint-based forgetting prevention of A-GEM is hypothesized to benefit from the representational alignment provided by distillation.
- SI + DER: Combines Synaptic Intelligence's online importance tracking with DER's logit-preserving replay. This creates a method that protects weights from two orthogonal angles: parameter space (SI) and output space (DER).

3.3 Dual Backbone Evaluation

The vast majority of continual learning papers use convolutional networks as the sole backbone. This project evaluates all methods on both Slim ResNet-18 and ViT-Small, enabling a principled study of whether method rankings and performance characteristics are architecture-dependent. Given the growing prominence of Vision Transformers in computer vision, understanding how

CL methods interact with attention-based backbones is a significant and underexplored research question.

3.4 Statistical Rigor

All experiments are run with 5 different random seeds per configuration, enabling:

- Paired t-tests for statistical significance between method pairs.
- Bonferroni correction for multiple comparison control across the full set of methods.
- Effect size analysis (Cohen's d) to quantify practical significance beyond p-values.
- Confidence intervals for all reported metrics.

This level of statistical rigor is rare in the continual learning literature, where single-seed results are the norm. It makes the results reproducible, interpretable, and trustworthy for practitioners.

3.5 Systematic Ablation Studies

The project includes systematic ablation studies that examine sensitivity to:

- Buffer size for replay methods: {100, 200, 500} samples per buffer slot.
- Regularization strength λ: {1, 10, 100, 1000} for EWC-based methods.
- Task sequence length: {5, 10} tasks on applicable datasets.

These ablations quantify the marginal contribution of each component in hybrid methods, identifying which combinations are synergistic and which are redundant. Such component-level analysis has not been performed for the novel hybrids studied in this project.

3.6 Data-Driven Decision Support Framework

The culminating contribution is an interactive Streamlit dashboard that takes a practitioner's resource constraints as inputs and recommends the most appropriate continual learning method. Inputs include:

- Available memory budget (replay buffer size constraint).
- Computational budget (training time per task constraint).
- Acceptable forgetting threshold (maximum F value).
- Task similarity expectations (Domain-IL vs. Class-IL scenario).

The recommendation engine is grounded in the empirical Pareto-optimality analysis performed over all experimental results — making it data-driven rather than heuristic-based. This is the first such tool for hybrid CL method selection to be grounded in a comprehensive, controlled empirical study.

3.7 Reproducible Open-Source Framework

All methods are implemented from scratch in PyTorch without relying on existing CL libraries (such as Avalanche or Mammoth), enabling transparent inspection of every algorithmic detail. The complete codebase includes:

- Dataset loaders for all 5 benchmark datasets.
- Model wrappers for all 3 backbone architectures with expandable classifier heads.
- Task-sequential training harness with task boundary handling.
- Metrics engine computing all 5 evaluation metrics automatically.
- Visualization pipeline generating heatmaps, bar charts, Pareto frontier plots, and training curves.
- YAML experiment configurations for all dataset-method combinations.
- Colab notebooks for large-scale experiments (Mini-ImageNet, Tiny-ImageNet) with FP16 and gradient checkpointing.

The full codebase will be released publicly with training logs, enabling the community to extend the comparison to new methods and datasets.

4. Algorithms and Methods

This section provides a structured overview of all algorithms implemented in the project, organized into baseline methods, hybrid methods, model architectures, and evaluation infrastructure.

4.1 Baseline Methods

Five baseline methods establish reference performance bounds. These span the spectrum from the worst-case to the oracle upper bound, providing the calibration points against which all CL methods are measured.

Method	Strategy	Data Stored	Role
Fine-Tuning	None	None	Lower bound — worst-case forgetting
Joint Training	Full history replay	All past data	Upper bound — oracle benchmark
EWC	Regularization (Fisher)	None	Regularization baseline
A-GEM	Gradient projection	Replay buffer	Efficient replay baseline
LwF	Knowledge distillation	None	Distillation-only baseline

4.1.1 Fine-Tuning (Lower Bound)

Standard sequential training with cross-entropy loss and no forgetting prevention. Achieves near-zero accuracy on all but the most recently learned task, demonstrating the severity of catastrophic forgetting and providing the absolute lower bound.

4.1.2 Joint Training (Upper Bound)

Trains simultaneously on all tasks seen so far — not a true CL method, but provides the oracle upper bound. The gap between any CL method and Joint Training quantifies the residual cost of the continual learning constraint.

4.1.3 Elastic Weight Consolidation (EWC)

Implements the Fisher Information-based weight protection described in Section 2.4.1. The diagonal FIM is computed from a held-out set at each task boundary. Regularization strength λ explored in $\{1, 10, 100, 1000\}$.

4.1.4 Averaged GEM (A-GEM)

Maintains a reservoir-sampled replay buffer. At each step, samples a batch to compute the episodic gradient and projects the current gradient onto the feasible region. Buffer sizes explored: {100, 200, 500}.

4.1.5 Learning without Forgetting (LwF)

Records model softmax outputs (at temperature T=2) before each task, uses them as distillation targets during training. Requires no replay buffer, making it the most memory-efficient baseline with explicit forgetting prevention.

4.2 Hybrid Methods

Seven hybrid methods combine two or more strategies. All use Slim ResNet-18 for local experiments; ViT-Small is used for large-scale Colab experiments.

Method	Components Combined	Key Innovation
DER	Replay + Distillation	Stores logits with samples for richer replay signal
X-DER	Replay + Distillation	Revises logits for Class-IL; reserves future class capacity
iCaRL	Replay + Distillation + Prototypes	Herding selection + nearest-mean classifier
ER-Reservoir	Replay only	Random reservoir — simple, competitive baseline
ER + EWC †	Replay + Regularization	Novel: replay corrects shift; EWC protects weights
A-GEM + Distill †	Gradient projection + Distillation	Novel: projection + representational alignment
SI + DER †	Reg. (online) + Replay + Distillation	Novel: parameter-space + output-space protection

† Novel hybrid combinations introduced in this project.

4.2.1 Dark Experience Replay (DER)

Stores input-logit pairs via reservoir sampling. Loss: $\mathscr{L}^{cl}$ (cross-entropy on current batch) + $\alpha\|f\theta(x^{mem}) - z^{mem}\|^2$ (MSE matching current predictions to stored logits). The stored logit vector encodes rich class-relationship information — the "dark experience" invisible in hard labels.

4.2.2 Extended DER (X-DER)

Extends DER with (i) a logit revision mechanism that updates stored predictions when the classifier head expands to accommodate new classes, and (ii) a future-class preparation mechanism. Essential for Class-IL where the output space is not fixed. State-of-the-art replay-distillation hybrid.

4.2.3 iCaRL

Three-component hybrid: herding-based exemplar selection (stores class-representative samples), knowledge distillation (preserves old output distributions), and nearest-mean-of-exemplars classification (non-parametric decision boundary). Particularly sample-efficient at small buffer sizes.

4.2.4 ER-Reservoir

Simplest possible replay: reservoir sampling with standard cross-entropy on the combined current + replayed batch. Serves as the critical "simple baseline" reference — how much does algorithmic complexity add over random replay?

4.2.5 ER + EWC (Novel)

Combines reservoir replay with EWC regularization: $\mathscr{L} = \mathscr{L}^{cl}(\text{current}) + \mathscr{L}^{cl}(\text{replay}) + \lambda \cdot \text{EWC penalty}$. Tests whether replay and weight-level regularization are synergistic (complementary) or redundant (replay already provides sufficient protection).

4.2.6 A-GEM + Distillation (Novel)

Augments A-GEM's gradient projection with a KL divergence distillation term. Hypothesis: gradient projection is necessary but not sufficient for forgetting prevention; distillation provides complementary representational alignment that projection alone cannot achieve.

4.2.7 SI + DER (Novel)

Combines Synaptic Intelligence's online weight importance tracking with DER's logit-preserving replay. Targets forgetting from two orthogonal angles simultaneously: parameter space (SI penalizes changes to important weights) and output space (DER forces current outputs to match stored outputs). Tests whether protection in both spaces compounds or saturates.

4.3 Model Architectures

Architecture	Parameters	VRAM (FP16)	Platform	Primary Use
ResNet-8	~80K	~0.5 GB	Local	Fast debug runs, MNIST-scale
Slim ResNet-18	~2M	~2 GB	Local + Kaggle	Main local backbone for all methods
ViT-Small	~22M	~6 GB	Colab / Kaggle	Transformer backbone for large-scale experiments

All backbones feature an expandable classifier head that grows as new classes arrive in Class-IL scenarios. ViT-Small uses gradient checkpointing and FP16 mixed precision to fit within Colab's 15 GB VRAM.

4.4 Datasets

Dataset	Scenario	Tasks	Classes/Task	Images	Platform
Permuted MNIST	Domain-IL	10	10	70K	Local
Split CIFAR-10	Class-IL	5	2	60K	Local
Split CIFAR-100	Class-IL	20	5	60K	Local (FP16)
Split Mini-ImageNet	Class-IL	20	5	60K	Colab / Kaggle
Sequential Tiny-ImageNet	Class-IL	10	20	120K	Colab / Kaggle

4.5 Preprocessing Pipeline

All datasets pass through a unified preprocessing pipeline with three stages:

4. Image Resizing: Normalizes input dimensions to match each backbone's required input shape (32×32 for local ResNet variants, 224×224 for ViT-Small).
5. Pixel Normalization: Standardizes pixel values to zero-mean, unit-variance using dataset-specific channel statistics, ensuring stable gradient descent across tasks.
6. Task-Specific Transformations: Applies fixed random pixel permutations to each Permuted MNIST task to generate distinct domain shifts while preserving digit class structure. FP16 tensor conversion is applied at the dataloader level for all VRAM-constrained experiments.

4.6 Evaluation Infrastructure

All experiments use a unified evaluation engine that computes all five metrics automatically after each task. The batch runner executes all method-dataset-seed combinations sequentially and exports results to timestamped CSV files. Results are aggregated into a master results file for statistical analysis.

Metric	Formula Summary	What It Measures
Average Accuracy (AA)	$(1/T)\ \Sigma\ a_{t,i}$	Overall retention across all tasks after final task
Forgetting (F)	$(1/(T-1))\ \Sigma\ [\max a_{j,i} - a_{t,i}]$	Average performance drop from peak; lower is better
Backward Transfer (BWT)	$(1/(T-1))\ \Sigma\ (a_{t,i} - a_{i,i})$	Negative = forgetting; positive = beneficial backward transfer
Forward Transfer (FWT)	$(1/(T-1))\ \Sigma\ (a_{i-1,i} - b_i)$	How much prior learning helps new tasks; higher is better
Time per Task	Seconds per task	Computational cost of each method

5. References

[1] Vaswani, A., et al. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30. https://arxiv.org/abs/1706.03762

[2] McCloskey, M., & Cohen, N. J. (1989). Catastrophic interference in connectionist networks: The sequential learning problem. Psychology of Learning and Motivation, 24, 109–165.

[3] French, R. M. (1999). Catastrophic forgetting in connectionist networks. Trends in Cognitive Sciences, 3(4), 128–135.

[4] Grossberg, S. (1980). How does a brain build a cognitive code? Psychological Review, 87(1), 1–51.

[5] Scialom, T., Chakrabarty, T., & Muresan, S. (2022). Fine-tuned language models are continual learners. EMNLP. https://arxiv.org/abs/2205.12393

[6] Wang, L., et al. (2024). A comprehensive survey of continual learning: Theory, method and application. IEEE TPAMI. https://arxiv.org/abs/2302.00487

[7] van de Ven, G. M., & Tolias, A. S. (2019). Three scenarios for continual learning. NeurIPS CL Workshop. https://arxiv.org/abs/1904.07734

[8] Kirkpatrick, J., et al. (2017). Overcoming catastrophic forgetting in neural networks. PNAS, 114(13), 3521–3526. https://arxiv.org/abs/1612.00796

[9] Zenke, F., Poole, B., & Ganguli, S. (2017). Continual learning through synaptic intelligence. ICML, 3987–3995. https://arxiv.org/abs/1703.04200

[10] Li, Z., & Hoiem, D. (2017). Learning without forgetting. IEEE TPAMI, 40(12), 2935–2947. https://arxiv.org/abs/1606.09282

[11] Lopez-Paz, D., & Ranzato, M. (2017). Gradient episodic memory for continual learning. NeurIPS, 30. https://arxiv.org/abs/1706.08840

[12] Chaudhry, A., et al. (2019). Efficient lifelong learning with A-GEM. ICLR. https://arxiv.org/abs/1812.00420

[13] Chaudhry, A., et al. (2019). On tiny episodic memories in continual learning. arXiv:1902.10486. https://arxiv.org/abs/1902.10486

[14] Buzzega, P., et al. (2020). Dark experience for general continual learning: A strong, simple baseline. NeurIPS, 33. https://arxiv.org/abs/2004.07211

[15] Boschini, M., et al. (2022). Class-incremental continual learning into the eXtended DER-verse. IEEE TPAMI. https://arxiv.org/abs/2201.00766

[16] Rebuffi, S.-A., et al. (2017). iCaRL: Incremental classifier and representation learning. CVPR. https://arxiv.org/abs/1611.07725

[17] Schwarz, J., et al. (2018). Progress & compress: A scalable framework for continual learning. ICML. https://arxiv.org/abs/1805.06370

[18] Rusu, A. A., et al. (2016). Progressive neural networks. arXiv:1606.04671. https://arxiv.org/abs/1606.04671

[19] Mallya, A., & Lazebnik, S. (2018). PackNet: Adding multiple tasks to a single network by iterative pruning. CVPR. https://arxiv.org/abs/1711.05769

[20] De Lange, M., et al. (2021). A continual learning survey: Defying forgetting in classification tasks. IEEE TPAMI, 44(7), 3366–3385. https://arxiv.org/abs/1909.08383

[21] Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. NIPS Deep Learning Workshop. https://arxiv.org/abs/1503.02531

[22] Geva, M., et al. (2021). Transformer feed-forward layers are key-value memories. EMNLP. https://arxiv.org/abs/2012.14913

[23] Dosovitskiy, A., et al. (2021). An image is worth 16×16 words: Transformers for image recognition at scale. ICLR. https://arxiv.org/abs/2010.11929

[24] Dey, J., et al. (2020). Simple lifelong learning machines. arXiv:2004.12908. https://arxiv.org/abs/2004.12908